KMIP on PKCS#11

Partial coverage — what is there, and why it stopped

The 18 features the gap matrix marks Partial, explained one by one

Document	Why each partially covered feature is partial
Features	18 of 82 assessed, across 7 domains
Grounded at	2964dee — the commit the gap matrix assesses
Reasons	11 unwritten · 3 built, never demonstrated · 1 deliberately stopped · 1 bounded by the token · 2 belongs to another product
Plan routing	13 closed by the delivery plan, 5 deferred
Date	12 September 2026

1 Why this document exists

Partial is the least informative verdict in the gap matrix. Covered and Not covered both tell a reader what to expect; Partial says something is there without saying how much, and hides a distinction that matters more than the verdict itself. A feature whose last ten per cent is scheduled and a feature whose remainder belongs to a different product are not in the same condition, and a buyer, an auditor and an engineer each need to tell them apart.

So every row below answers the same four questions, always in the same order:

- **What exists —** the part that is built, and where it lives in the repository.
- **What is missing —** the part that is not, stated as a capability rather than a task.
- **Why it stopped here —** the actual reason — which is sometimes a decision, sometimes a device, and sometimes nothing more interesting than nobody having done it.
- **What would complete it —** the work, and the test that would settle it.

A note on how these claims were checked

The coverage verdicts come from the gap matrix, which was assessed against the repository at 2964dee with the full suite green. The explanations here were grounded by reading the code and citing it — the file and line references are real and current. They were not re-verified by running the suite: the container this document was generated in has no SoftHSM installation, so the tests, which need a live token, could not execute. That is a limit on this document, not on the assessment it describes, and it is recorded here rather than left for a reader to discover.

2 Five reasons a feature stops

These are the categories the eighteen sort into. They are ordered by how likely each is to change: the first moves with a week of work, the last will not move at all.

Reason	Means	Count	Outlook
Unwritten	The remainder is ordinary work that nobody has done yet. Nothing blocks it.	11	Moves when someone writes it
Built, never demonstrated	The code exists; the proof does not. Until it runs against the real thing it is a claim, not a capability.	3	Moves when it is demonstrated
Deliberately stopped	Going further would cost more than it returns, or would undo something worth keeping.	1	Moves only if the trade-off is revisited
Bounded by the token	The limit belongs to the PKCS#11 device, not to this code. A different token moves it with no change here.	1	Moves with different hardware
Belongs to another product	What exists is the part that fits a key manager. The rest is a different thing wearing the same word.	2	Does not move; the boundary is the answer

The distribution is the finding. Most of these features are not partial because of a hard problem or a considered trade-off — they are partial because the work stopped at the point where the phase that introduced them ended. That is worth saying plainly: it means most of this column is schedulable, and it means the small number of genuinely immovable rows deserve more attention than their count suggests.

3 The eighteen at a glance

Feature	Domain	Reason	Where the plan puts it
Automatic rotation	Key lifecycle and cryptography	**Unwritten**	Step 5 — Write endpoints, and the governance gaps (stage B)
Algorithm breadth	Key lifecycle and cryptography	**Bounded by the token**	*Deferred — supplier*
Split knowledge / M-of-N shares	Key lifecycle and cryptography	**Unwritten**	Step 5 — Write endpoints, and the governance gaps (stage B)
Bulk and batch operations	Key lifecycle and cryptography	**Unwritten**	Step 5 — Write endpoints, and the governance gaps (stage B)
Database TDE integration	Protocol and ecosystem integration	**Built, never demonstrated**	*Deferred — validation*
Storage, backup and VM integrations	Protocol and ecosystem integration	**Built, never demonstrated**	*Deferred — validation*
Secrets management	Protocol and ecosystem integration	**Belongs to another product**	*Deferred — product scope*
Certificate lifecycle	Protocol and ecosystem integration	**Belongs to another product**	*Deferred — product scope*
Encryption as a service (data-plane API)	Protocol and ecosystem integration	**Unwritten**	Step 5 — Write endpoints, and the governance gaps (stage B)
Separation of duties	Identity and access control	**Unwritten**	Step 6 — Administration, OpenAPI, separation of duties (stage B)
Key inventory and discovery	Audit, compliance and assurance	**Unwritten**	Step 2 — Query and reporting layer (stage A)
Crypto-agility reporting	Audit, compliance and assurance	**Unwritten**	Step 2 — Query and reporting layer (stage A)
Horizontal throughput	Availability, scale and recovery	**Deliberately stopped**	Step 11 — High availability, failover and deployment (stage D)
Scheduled and offsite backup	Availability, scale and recovery	**Unwritten**	Step 11 — High availability, failover and deployment (stage D)
Kubernetes / container deployment	Availability, scale and recovery	**Built, never demonstrated**	Step 11 — High availability, failover and deployment (stage D)
Alerting	Operations and observability	**Unwritten**	Step 9 — Audit externalisation, compliance and ceremony (stage C)
Request bounds and DoS resistance	Platform hardening	**Unwritten**	Step 3 — HTTP transport, sessions and service accounts (stage B)
Sealed startup with quorum unseal	Platform hardening	**Unwritten**	Step 9 — Audit externalisation, compliance and ceremony (stage C)

The routing column is read from the delivery plan at build time, not written here. If the plan stops closing one of these, this document refuses to build rather than continuing to promise it.

4 Feature by feature

Key lifecycle and cryptography

1. Automatic rotation

UNWRITTEN · *Step 5 — Write endpoints, and the governance gaps (stage B)*

The market expects Expiring keys replaced automatically, with lineage preserved.

What exists A background scheduler in lifecycle/governance.py scans for keys reaching their cryptoperiod, creates the replacement inside the token, and cross-links the pair so the lineage is identical to a client-driven ReKey. Every action is audited as system:scheduler.

What is missing Key pairs. _rotate() raises on any object that is not a SymmetricKey (governance.py:157), so an expiring RSA or EC key is reported and left alone rather than replaced.

Why it stopped here The scheduler was built in Phase 5 against the case that mattered first — data-encryption keys with short cryptoperiods. Asymmetric rotation needs no new thinking: ReKeyKeyPair already exists and produces the same links. It was scope, not difficulty.

What would complete it Call the existing ReKeyKeyPair handler from _rotate() for key-pair objects and cross-link both new objects to both old ones. The test that matters is the lineage, not the rotation.

2. Algorithm breadth

BOUNDED BY THE TOKEN · *Deferred — supplier*

The market expects AES, RSA, EC, and the curve and hash families an enterprise estate actually uses.

What exists The shim reads the token's real mechanism list once at startup (pkcs11_shim/shim.py:222) and gates every operation on it (shim.py:238), so an unsupported request fails cleanly at the edge instead of surfacing a raw PKCS#11 error from inside an operation.

What is missing Roughly two-thirds of the KMIP algorithm enum. SoftHSM2 2.7.0 advertises 79 mechanisms; the packaged 2.6.1 advertises 70 and lacks CKM_ECDSA_SHA256 entirely, which is why this project builds 2.7.0 from source.

Why it stopped here This is the clearest case in the document of a limit that is not the code's. Nothing in the KMIP layer refuses an algorithm the token can perform. The gap is the device.

What would complete it A richer token. The probe activates whatever it finds, so breadth arrives with the hardware and no code changes. Treat this row as a procurement line, not a backlog item.

3. Split knowledge / M-of-N shares

UNWRITTEN · *Step 5 — Write endpoints, and the governance gaps (stage B)*

The market expects Key material split so no one custodian can reconstruct it.

What exists CreateSplitKey produces XOR shares of key material, and the object model, state machine and access control treat the parts like any other managed object.

What is missing Threshold recovery. The implementation is strictly N-of-N (create_split_key.py:40) — every share is needed, so losing one custodian loses the key. Any SplitKeyMethod other than XOR is refused outright (create_split_key.py:35).

Why it stopped here XOR is the split that takes ten lines and no library. Shamir needs finite-field arithmetic and a careful implementation, and the value only appears when there is an operational custodian process to use it.

What would complete it Shamir sharing behind the existing SplitKeyMethod enum — the protocol already has the vocabulary. The test is the one that defines the feature: any k of n shares reconstruct, any k-1 do not.

4. Bulk and batch operations

UNWRITTEN · *Step 5 — Write endpoints, and the governance gaps (stage B)*

The market expects Many operations per round trip, for provisioning at scale.

What exists A request carrying many Batch Items is decoded and each item dispatched in turn (server/server.py:380), with per-item result status, so the round-trip saving a bulk client wants is real.

What is missing Atomicity. Each dispatch commits on its own, so an item that fails halfway through a batch leaves everything before it applied and everything after it not.

Why it stopped here KMIP defines no rollback — the specification has batch semantics but no transaction, so there is nothing on the wire to be compliant with. That made it easy to leave, and it is the kind of gap that stays invisible until a provisioning run half-succeeds.

What would complete it A transaction around the batch in the service layer rather than in the protocol, with the boundary chosen explicitly: a batch of reads should not take a write lock.

Protocol and ecosystem integration

5. Database TDE integration

BUILT, NEVER DEMONSTRATED · *Deferred — validation*

The market expects Oracle, SQL Server, MongoDB, Db2 taking keys from the KMS.

What exists Oracle, SQL Server, MongoDB and Db2 obtain keys over KMIP, and this server implements the operations their profiles use. The path is there in the sense that nothing is known to be missing.

What is missing Evidence. No database has ever been pointed at this server.

Why it stopped here Interoperability is not a property of a specification, it is a property of two implementations that have met. Every vendor profile makes assumptions the text does not state, and they surface on first contact rather than on reading.

What would complete it One vendor at a time: run the database against the server, fix what the profile assumes, and add the exchange to the conformance suite so it stays fixed.

6. Storage, backup and VM integrations

BUILT, NEVER DEMONSTRATED · *Deferred — validation*

The market expects VMware, NetApp, Veeam, tape libraries.

What exists The same position as database TDE. VMware, NetApp, Veeam and tape libraries are KMIP clients and the operations they need are implemented.

What is missing The same thing: a demonstration. None has been connected.

Why it stopped here Identical reasoning, and grouped separately only because the buyers are different. Storage profiles tend to lean on attributes and Locate filters more heavily than database profiles do, so the first contact is likely to find different assumptions.

What would complete it Validation against one product in each family, with the exchanges captured as tests.

7. Secrets management

BELONGS TO ANOTHER PRODUCT · *Deferred — product scope*

The market expects Arbitrary secrets with versioning, leases and dynamic issuance.

What exists SecretData objects hold arbitrary secrets with the full lifecycle, access control and audit that keys get, and ObtainLease covers time-bounded custody.

What is missing Dynamic issuance. There is no engine that mints a database credential on request, no templating, and no automatic revocation when a lease ends.

Why it stopped here Static custody is a key manager's job and it is done. Dynamic secrets are a different product — the engine has to understand each backend well enough to create and revoke principals in it, which is an integration surface, not a cryptographic one.

What would complete it Nothing, in this product. The honest route is to run a secrets manager beside this server and let it use this one for the keys it needs. Building a second Vault inside a KMS serves neither.

8. Certificate lifecycle

BELONGS TO ANOTHER PRODUCT · *Deferred — product scope*

The market expects CA issuance, CSR handling, renewal, revocation checking.

What exists Certify, ReCertify and Validate are implemented, certificates are first-class managed objects, and the private key never leaves the token during signing.

What is missing Everything that makes a CA a CA: no chain to a real issuer, no CRL, no OCSP, no policy or profile enforcement. What comes out is self-signed.

Why it stopped here KMIP's Certify was never a certificate authority and should not be made into one. A CA is a governed, audited, long-lived trust anchor with its own operational discipline; reimplementing a weak one inside a key manager would invite exactly the reliance it could not support.

What would complete it Front the deployment with a real CA — EJBCA or equivalent — and let this server hold the keys. The boundary is the point.

9. Encryption as a service (data-plane API)

UNWRITTEN · *Step 5 — Write endpoints, and the governance gaps (stage B)*

The market expects Applications send data rather than fetching keys: encrypt, decrypt, sign and HMAC, plus data-key issuance and re-wrap to a newer key version.

What exists Encrypt, Decrypt, Sign, MAC and Hash all execute inside the token over KMIP, so an application can already use the service without ever holding key material. The hard half is done.

What is missing The shapes applications actually consume: a data-key call returning the wrapped and plaintext forms together, re-wrap to a newer key version, and any of it over HTTP.

Why it stopped here The primitives were built protocol-first, against the KMIP specification. The convenience operations that make encryption as a service pleasant to use are conventions of REST key managers, not KMIP operations, so they never arrived with the protocol work.

What would complete it Data-key and re-wrap in the service layer, exposed on both transports. The test worth insisting on is cross-transport: ciphertext from the HTTP endpoint must decrypt over KMIP.

Identity and access control

10. Separation of duties

UNWRITTEN · *Step 6 — Administration, OpenAPI, separation of duties (stage B)*

The market expects Key administrators cannot use the keys they administer.

What exists Ownership, per-object grants, groups and per-role operation allowlists are all enforced, and dual control prevents a single identity completing a Destroy or an Export alone.

What is missing A split administrator. The reserved admin role short-circuits every check (lifecycle/access_control.py:53), so one identity can both grant access to a key and use it.

Why it stopped here The admin role was the first thing built, when the alternative was no access control at all, and everything since has been layered around it. Splitting it is not hard — it is a migration, which is why it did not happen incidentally.

What would complete it Two roles where there is one: security-admin holding identities, roles and policy, key-admin holding objects and cryptoperiods, with neither able to become the other. Existing admins get both on upgrade so nobody's access changes until an operator splits them deliberately.

Audit, compliance and assurance

11. Key inventory and discovery

UNWRITTEN · *Step 2 — Query and reporting layer (stage A)*

The market expects What keys exist, who owns them, what state they are in.

What exists Locate answers inventory questions over the wire with attribute filters, and kmip-admin lists objects for an operator.

What is missing A report. Nothing aggregates, pages, sorts or exports, so answering "what expires this quarter" means a client program.

Why it stopped here Locate is a protocol operation and does what the protocol asks. Reporting is a console feature, and there is no console — this row is partial largely because the REST layer it would belong to does not exist yet.

What would complete it Paged, sortable queries in the store returning whole rows rather than bare identifiers, with cursors rather than offsets so a table being written does not double-show rows.

12. Crypto-agility reporting

UNWRITTEN · *Step 2 — Query and reporting layer (stage A)*

The market expects Which algorithms are in use and where, for migration planning.

What exists Both halves of the data are present: the capability probe knows what the token supports, and every managed object records its algorithm and parameters.

What is missing The join. Nothing counts algorithms in use, flags the weak ones or estimates the work in a migration.

Why it stopped here The same reason as inventory reporting, and the same fix. The information has been sitting in the store since Phase 1 with no surface to present it on.

What would complete it An aggregate query and a report, once there is somewhere to show it. This is the row most likely to matter in a post-quantum migration, which is worth remembering before deprioritising it.

Availability, scale and recovery

13. Horizontal throughput

DELIBERATELY STOPPED · *Step 11 — High availability, failover and deployment (stage D)*

The market expects Scale with cores and nodes.

What exists A pre-fork worker pool spreads connections across processes, and the scrypt verification cache lifted a single worker from about 24 to roughly 600 operations per second.

What is missing Linear scaling. Measured gain is about 1.5×, not one per core, and adding workers past that returns little.

Why it stopped here The bound is the audit chain. Each entry hashes its predecessor, so entries must be written in order and the write lock serialises them. That is not a misconfiguration — it is the price of a log that can prove it has not been edited, and it was accepted knowingly.

What would complete it Per-shard chains with a periodic cross-link, or an external append-only service. Both trade some of the tamper-evidence property for throughput, so the honest framing is a choice rather than a fix.

How the plan closes it anyway Step 11 closes this row without taking that trade: it adds nodes rather than weakening the chain. Horizontal throughput arrives through clustering, and the single-node ceiling of about 1.5× stays exactly where it is. Worth knowing if the deployment is ever one machine — there, this row does not move.

14. Scheduled and offsite backup

UNWRITTEN · *Step 11 — High availability, failover and deployment (stage D)*

The market expects Automatic snapshots shipped off the box.

What exists Online backup while the server runs, paired to the token that can decrypt it, with a restore that verifies what it restored rather than reporting success on exit status.

What is missing A scheduler and a shipper. When a backup runs and where it lands are the operator's to arrange.

Why it stopped here The original reasoning was that a KMS growing its own cron and its own object-store client duplicates what every platform already has. That holds for a single box and stops holding for a cluster, where a backup has to be taken somewhere and restorable somewhere else — so the plan takes the scheduling on rather than leaving it out.

What would complete it A scheduler and an offsite shipper, built alongside the clustering work because that is what makes them necessary. The test is the one that matters operationally and is easy to skip: a backup taken on one node restores on another.

15. Kubernetes / container deployment

BUILT, NEVER DEMONSTRATED · *Step 11 — High availability, failover and deployment (stage D)*

The market expects A supported image and manifests.

What exists A two-stage Dockerfile, a systemd unit, configuration validation with --check, and health and readiness endpoints suitable for probes. The inputs are checked at build time.

What is missing Any evidence it runs, plus a Helm chart or operator. The image has never been executed: this environment blocks Docker Hub and the SoftHSM2 mirror the build needs.

Why it stopped here Environment. This is the one row in the document that is unproven because of where the work happened rather than what was chosen. Saying so is the point — an unexecuted Dockerfile is a guess with good syntax.

What would complete it Build and run the image somewhere with network access, then a chart carrying the PKCS#11 device, the readiness probe and the single-writer constraint the audit chain imposes.

Operations and observability

16. Alerting

UNWRITTEN · *Step 9 — Audit externalisation, compliance and ceremony (stage C)*

The market expects Expiry and anomaly alerts pushed to on-call.

What exists Approaching expiry is logged and audited, operation counters and authentication failures are exported for Prometheus, and the JSON log carries enough structure to alert on.

What is missing Anything that pushes. Every signal waits to be collected; nothing reaches a person.

Why it stopped here The signals were built with the metrics endpoint in Phase 3 and the scheduler in Phase 5, each for its own reason, and nobody joined them to a notifier. There is no design decision behind this row — it is simply unfinished.

What would complete it Alertmanager rules over the metrics already exported closes most of it without new code. What does need writing is expiry alerting that fires once per key per window rather than on every scheduler scan.

Platform hardening

17. Request bounds and DoS resistance

UNWRITTEN · *Step 3 — HTTP transport, sessions and service accounts (stage B)*

The market expects Hostile input cannot exhaust the server.

What exists Request size is capped at one mebibyte before the body is read (server/server.py:38), TTLV nesting is bounded at 32 levels (core/ttlv.py:19) so a recursive structure cannot exhaust the stack, and the TLS handshake happens off the accept loop under a timeout.

What is missing A cap on how many clients may connect and how fast any one of them may ask. The listen backlog — 16 in the single-process server, 128 under the worker pool — is a queue depth, not a limit: it governs how many connections wait to be accepted, not how many are served or how often.

Why it stopped here Each existing bound was added against a specific attack — an oversized body, a nesting bomb, a slow handshake. Nobody added the general one, and a backlog number sitting in the code reads like a limit without being one.

What would complete it A bounded connection semaphore and a per-identity token bucket returning 429 or its KMIP equivalent, tested by holding two hundred connections open and confirming the server still serves.

18. Sealed startup with quorum unseal

UNWRITTEN · *Step 9 — Audit externalisation, compliance and ceremony (stage C)*

The market expects The service starts sealed and its state stays unreadable until unsealed — by a quorum of operator-held shares, or automatically by a trusted device — and the shares can be re-issued without downtime.

What exists The auto-unseal half, and it is genuine: the metadata store is encrypted under a non-extractable HSM master key, so a stolen database file is inert without the token. Master-key rotation is online and row-by-row.

What is missing The operator half. There is no quorum — whoever holds the token PIN starts the service alone — and no ceremony for re-issuing shares.

Why it stopped here The threat this project designed against was a stolen database, and against that the token is a complete answer. The threat a quorum addresses is a single trusted operator, which is an organisational control, and it was never in scope.

What would complete it Split the PIN into operator-held shares so a quorum starts the service. The behaviour to get right is the sealed state itself: a sealed server should answer its health probe and refuse every operation, not fail obscurely inside the shim.

5 What to take from it

If you are deciding whether to depend on this server

- The rows marked Belongs to another product will not move, and that is the answer rather than a shortfall: run a CA and a secrets manager beside this server rather than expecting it to become them.
- The row marked Bounded by the token is a procurement question. Algorithm breadth arrives with the hardware and needs no change here.
- The rows marked Built, never demonstrated are the ones to ask about before committing. An untested integration path is a plan, not a capability, and the project says so rather than counting it.

If you are planning the work

- 11 of the 18 are simply unwritten, and the delivery plan schedules most of them. None needs a new design.
- Horizontal throughput is the one row where completing it means giving something up. Lifting it past about 1.5× means weakening the single serial audit chain, so treat it as a decision to be taken deliberately rather than a performance bug to be fixed.
- Separation of duties is the smallest change with the largest assurance return: one role becomes two, and the claim that no administrator can both grant access and use it becomes true.

One thing this document deliberately does not do is round upward. Every row here could be described as covered with a caveat; none is. A feature is partial until the thing a reader would assume from the word "covered" is actually true.